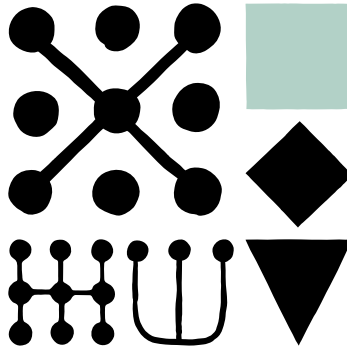


我们如何构建多 Agent 研究系统

How we built our multi-agent research system

Anthropic · 2025-06-13 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



我们的 Research 功能使用多个 Claude Agent，更有效地探索复杂主题。本文分享我们构建这个系统时遇到的工程挑战，以及从中学到的经验。

Claude 现在具备了 Research 能力，可以在网络、Google Workspace 及各种集成服务中搜索，以完成复杂任务。

这个多 Agent 系统从原型走向生产的过程，让我们在系统架构、工具设计和提示词工程方面积累了重要经验。多 Agent 系统由多个协同工作的 Agent 组成，这些 Agent 是在循环中自主使用工具的大语言模型。我们的 Research 功能中有一个 Agent，会根据用户查询规划研究过程，然后通过工具创建并行 Agent，让它们同时搜索信息。引入多个 Agent，也给 Agent 之间的协调、评测和可靠性带来了新的挑战。

本文将逐一介绍对我们行之有效的原则，希望这些经验也能帮助你构建自己的多 Agent 系统。

多 Agent 系统的优势

研究工作涉及开放式问题，所需步骤很难事先预测。探索复杂主题时，无法把一条固定路径硬编码下来，因为这个过程本质上是动态的，而且依赖此前走过的路径。人们开展研究时，往往会根据新发现不断调整方法，追踪调查过程中浮现的线索。

这种不可预测性使 AI Agent 特别适合研究任务。随着调查推进，研究需要灵活改变方向，或探索旁支联系。模型必须连续自主运行许多轮，根据中间发现决定下一步追踪哪些方向。线性的、一次性执行的流水线无法应对这些任务。

搜索的本质是压缩：从庞大的资料库中提炼洞见。子 Agent 分别使用自己的上下文窗口并行运行，同时探索问题的不同方面，再把最重要的 token 凝练后交给主研究 Agent，从而帮助实现压缩。每个子 Agent 也实现了关注点分离，拥有不同的工具、提示词和探索轨迹。这减少了路径依赖，使深入、独立的调查成为可能。

当智能达到某个门槛后，多 Agent 系统就成了提升性能的重要途径。例如，尽管过去 10 万年间人类个体变得更聪明了，但在信息时代，人类社会能力的指数级增长，靠的是我们的集体智慧与协调能力。即便具备通用智能，Agent 独自行动时仍然会受到限制；一组 Agent 能完成的事情要多得多。

我们的内部评测表明，多 Agent 研究系统尤其擅长广度优先的查询，这类查询需要同时推进多个相互独立的方向。我们发现，在内部研究评测中，以 Claude Opus 4 为主 Agent、Claude Sonnet 4 为子 Agent 的多 Agent 系统，性能比单 Agent 的 Claude Opus 4 高出 90.2%。例如，面对“找出标普 500 指数中信息技术板块所有公司的全部董事会成员”这一要求，多 Agent 系统把它拆成子 Agent 的任务后找到了正确答案；而单 Agent 系统以缓慢、串行的方式搜索，未能找到答案。

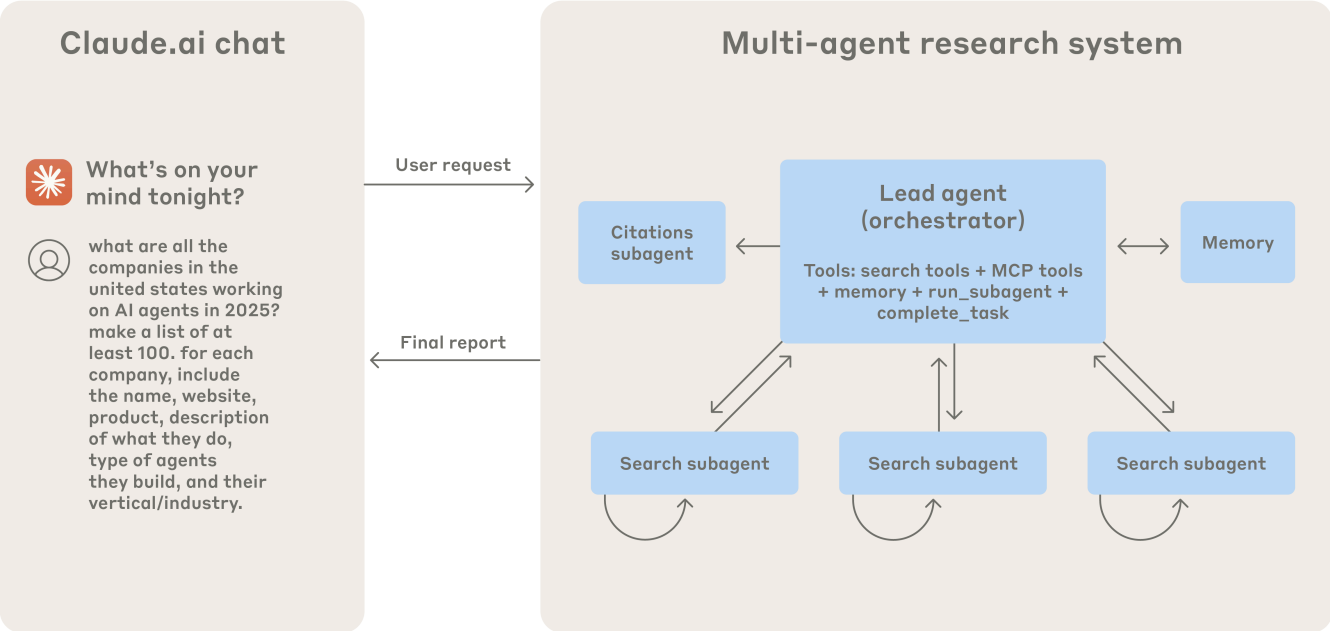
多 Agent 系统之所以有效，主要是因为它们有助于投入足够的 token 来解决问题。在我们的分析中，三个因素解释了 BrowseComp 评测中 95% 的性能方差。该评测测试的是浏览 Agent 找到难以获取的信息的能力。我们发现，仅 token 用量就能解释 80% 的方差，另外两个具有解释力的因素是工具调用次数和模型选择。这一发现验证了我们的架构：把工作分配给具有独立上下文窗口的多个 Agent，从而增加并行推理的容量。最新的 Claude 模型极大地提高了 token 使用效率，因为升级到 Claude Sonnet 4 带来的性能提升，比把 Claude Sonnet 3.7 的 token 预算翻倍还要大。对于超出单个 Agent 能力上限的任务，多 Agent 架构能够有效扩大 token 投入。

但也有一个缺点：在实际运行中，这类架构消耗 token 的速度很快。我们的数据显示，Agent 的 token 用量通常约为聊天交互的 4 倍，多 Agent 系统则约为聊天的 15 倍。要在经济上可行，任务本身的价值必须足够高，才能覆盖多 Agent 系统提升性能所增加的成本。此外，某些领域要求所有 Agent 共享同一份上下文，或 Agent 之间存在大量依赖关系，目前并不适合采用多 Agent 系统。例如，与研究相比，大多数编程任务中真正可以并行开展的工作较少，而且大语言模型 Agent 目前还不擅长实时协调其他 Agent 并向其委派任务。我们发现，多 Agent 系统擅长处理那些价值较高、需要大量并行工作、信息量超出单个上下文窗口、并且要对接众多复杂工具的任务。

Research 架构概览

我们的 Research 系统采用多 Agent 架构，使用编排者—工作者模式：由主 Agent 协调整个过程，同时把工作委派给并行运行的专门子 Agent。

High-level Architecture of Advanced Research

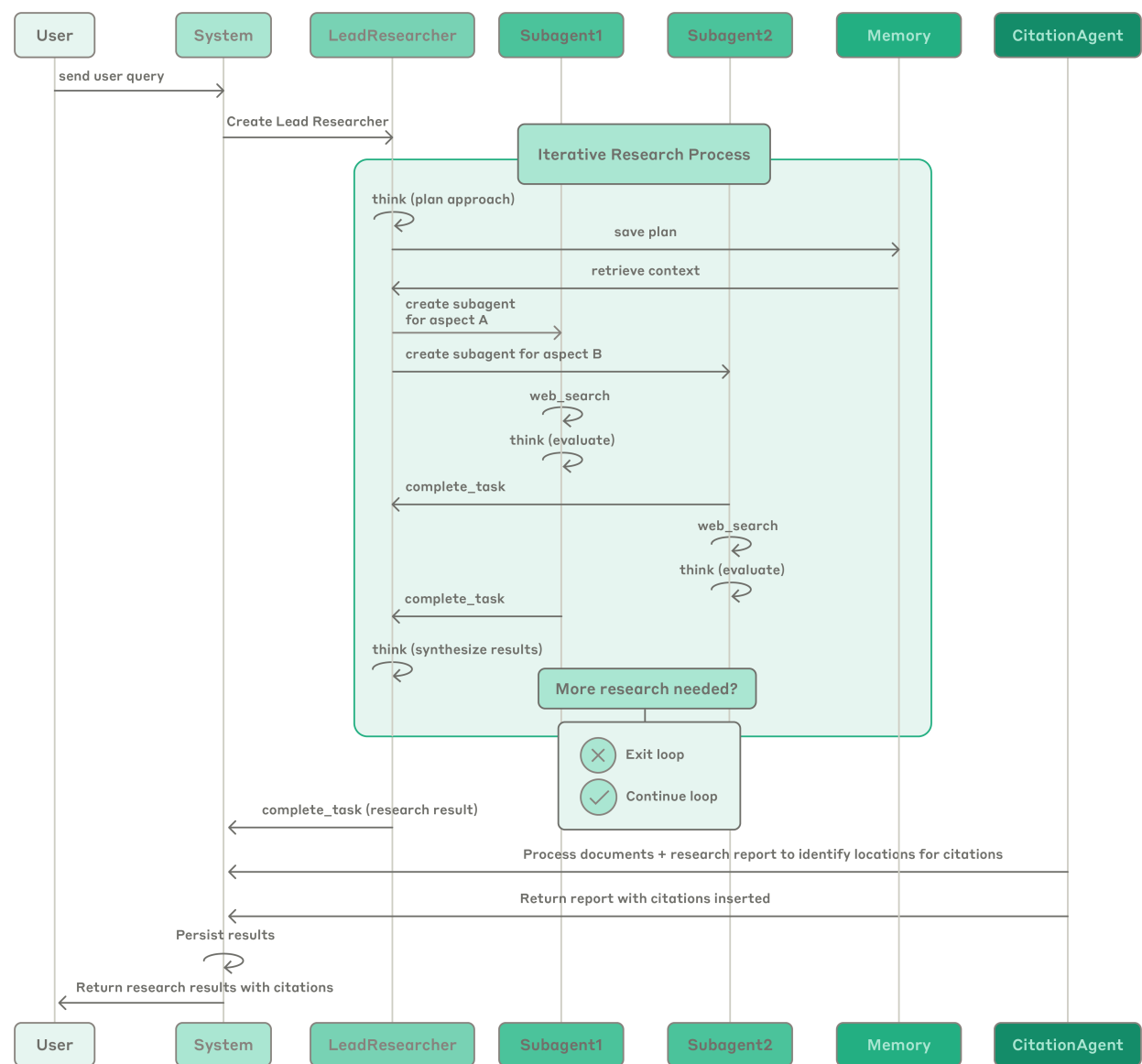


The multi-agent architecture in action: user queries flow through a lead agent that creates specialized subagents to search for different aspects in parallel.

用户提交查询后，主 Agent 会分析查询、制定策略，并创建子 Agent，让它们同时探索不同方面。如上图所示，子 Agent 充当智能过滤器，反复使用搜索工具收集信息。在这个例子中，它们收集的是 2025 年 AI Agent 公司的信息，然后把公司名单返回给主 Agent，由主 Agent 汇总成最终答案。

传统的检索增强生成（RAG）方法采用静态检索：获取与输入查询最相似的一组文本片段，再利用这些片段生成回答。我们的架构则采用多步搜索，动态发现相关信息，根据新发现调整方向，并分析结果以形成高质量答案。

Multi-agent System Process Diagram



Process diagram showing the complete workflow of our multi-agent Research system. When a user submits a query, the system creates a LeadResearcher agent that enters an iterative research process. The LeadResearcher begins by thinking through the approach and saving its plan to Memory to persist the context, since if the context window exceeds 200,000 tokens it will be truncated and it is important to retain the plan. It then creates specialized Subagents (two are shown here, but it can be any number) with specific research tasks. Each Subagent independently performs web searches, evaluates tool results using interleaved thinking, and returns findings to the LeadResearcher. The LeadResearcher synthesizes these results and decides whether more research is needed—if so, it can create additional subagents or refine its strategy. Once sufficient information is gathered, the system exits the research loop and passes all findings to a CitationAgent, which processes the documents and research report to identify specific locations for citations. This ensures all claims are properly attributed to their sources. The final research results, complete with citations, are then returned to the user.

研究 Agent 的提示词工程与评测

多 Agent 系统与单 Agent 系统存在一些关键差异，其中之一就是协调复杂度会迅速增长。早期的 Agent 曾出现这样的错误：为一个简单查询创建 50 个子 Agent，为寻找根本不存在的来源而无休止地搜索网络，以及用过多的进展更新干扰彼此。由于每个 Agent 都受提示词引导，提示词工程成了我们改善这些行为的主要手段。下面是在我们为 Agent 编写提示词时学到的一些原则：

- 1. 像你的 Agent 一样思考。**要迭代提示词，必须理解它们的实际影响。为此，我们使用 Console 搭建了模拟环境，采用与系统完全相同的提示词和工具，然后逐步观察 Agent 如何工作。这立刻暴露出一些失败模式：已经获得足够结果却还在继续、搜索查询过于冗长，或选择了错误工具。有效的提示词依赖于对 Agent 建立准确的心智模型，这能让最有影响力的改动变得显而易见。
- 2. 教会编排者如何委派任务。**在我们的系统中，主 Agent 会把查询拆分成子任务，并向子 Agent 描述这些任务。每个子 Agent 都需要明确的目标、输出格式、对工具和信息来源的使用指导，以及清晰的任务边界。缺少详细的任务描述，Agent 就会重复劳动、遗漏工作，或无法找到必要信息。最初，我们允许主 Agent 给出简短的指令，例如“研究半导体短缺”，但发现这些指令往往过于模糊，导致子 Agent 误解任务，或与其他 Agent 执行完全相同的搜索。例如，一个子 Agent 调查的是 2021 年汽车芯片危机，另外两个则重复调查当前的 2025 年供应链，未能形成有效分工。
- 3. 让投入规模与查询复杂度相匹配。**Agent 难以判断不同任务应投入多少精力，因此我们在提示词中加入了投入规模规则。简单的事实查找只需 1 个 Agent，调用工具 3—10 次；直接比较可能需要 2—4 个子 Agent，每个调用工具 10—15 次；复杂研究则可能需要超过 10 个职责明确的子 Agent。这些明确指引帮助主 Agent 高效分配资源，避免在简单查询上投入过多，而后者是早期版本中常见的失败模式。
- 4. 工具设计与选择至关重要。**Agent 与工具之间的接口，与人机界面同样关键。使用正确的工具能提高效率，而且往往是必不可少的。例如，如果所需上下文只存在于 Slack 中，Agent 却去搜索网络，那么它从一开始就注定会失败。通过 MCP 服务器让模型访问外部工具后，这个问题会更加突出，因为 Agent 会遇到从未见过的工具，而工具描述的质量参差不齐。我们为 Agent 提供了明确的启发式方法，例如先检查所有可用工具，让工具使用方式匹配用户意图，进行广泛的外部探索时搜索网络，以及优先使用专用工具而非通用工具。糟糕的工具描述可能把 Agent 引向完全错误的方向，因此每个工具都需要有独特的用途和清晰的描述。
- 5. 让 Agent 自我改进。**我们发现，Claude 4 系列模型可以成为出色的提示词工程师。给它们一段提示词和一种失败模式，它们就能诊断 Agent 失败的原因，并提出改进建议。我们甚至创建了一个工具测试 Agent：给它一个有缺陷的 MCP 工具后，它会尝试使用这个工具，然后重写工具描述以避免失败。通过数十次测试，它发现了一些关键细节和缺陷。这个改善工具易用性的过程，让后续使用新描述的 Agent 将任务完成时间缩短了 40%，因为它们能够避免大多数错误。
- 6. 先广泛探索，再逐步聚焦。**搜索策略应当效仿研究专家：先了解全貌，再深入具体问题。Agent 往往默认使用过长、过于具体的查询，结果很少。为克服这一倾向，我们在提示词中要求 Agent 先用简短、宽泛的查询起步，评估可获得的信息，再逐渐缩小范围。
- 7. 引导思考过程。**扩展思考模式会让 Claude 在可见的思考过程中输出额外 token，可以充当一块可控的草稿板。主 Agent 利用思考来规划方法，评估哪些工具适合任务、确定查询复杂度和子 Agent 数量，并定义各个子 Agent 的职责。我们的测试表明，扩展思考改善了指令遵循、推理和效率。子 Agent 也会先规划，再在工具返回结果后使用交错思考来评估质量、识别缺口并改进下一次查询。这使子 Agent 能更有效地适应各种任务。

8. **并行工具调用显著改善速度与性能。**复杂研究任务天然需要探索许多来源。早期 Agent 采用串行搜索，速度慢得令人难以忍受。为加快速度，我们引入了两种并行化方式：(1) 主 Agent 并行启动 3—5 个子 Agent，而非串行启动；(2) 子 Agent 并行使用至少 3 个工具。对于复杂查询，这些改动将研究时间最多缩短了 90%，让 Research 能在几分钟而非几小时内完成更多工作，同时覆盖比其他系统更多的信息。

我们的提示词策略着重于让 Agent 掌握有效的启发式方法，而非僵硬的规则。我们研究了熟练的人类研究者如何处理研究任务，并将这些策略写入提示词，例如把难题拆成较小的任务、仔细评估来源质量、根据新信息调整搜索方法，以及判断何时应该重视深度（详细调查一个主题），何时应该重视广度（并行探索多个主题）。我们也通过设置明确的约束，主动减轻意外副作用，防止 Agent 失控。最后，我们着力建立由可观测性和测试用例支持的快速迭代循环。

有效评测 Agent

构建可靠的 AI 应用离不开良好的评测，Agent 也不例外。不过，评测多 Agent 系统面临独特挑战。传统评测往往假定 AI 每次都遵循相同的步骤：给定输入 X，系统应沿着路径 Y 产生输出 Z。但多 Agent 系统并不是这样运行的。即使起点完全一致，Agent 也可能通过截然不同但同样有效的路径达成目标。一个 Agent 可能搜索三个来源，另一个则搜索十个；它们也可能使用不同工具找到同一个答案。由于我们并不总是知道哪些步骤才是正确的，通常不能仅仅检查 Agent 是否遵循了预先规定的“正确”步骤。我们需要灵活的评测方法，既判断 Agent 是否取得正确结果，也判断它是否遵循了合理的过程。

立即从小样本开始评测。在 Agent 开发早期，由于存在大量容易取得的改进，改动往往会带来显著影响。一次提示词调整可能把成功率从 30% 提高到 80%。效果差异如此之大时，只需几个测试用例就能看出变化。我们最初使用了一组大约 20 个查询，代表真实的使用模式。测试这些查询，往往就能清楚看到改动的影响。我们经常听到 AI 开发团队说，他们推迟建立评测，是因为认为只有包含数百个测试用例的大规模评测才有用。不过，最好立即用几个示例开展小规模测试，而不是等到能够建立更全面的评测时再开始。

方法得当时，以大语言模型为裁判的评测可以扩展规模。研究输出是自由形式的文本，而且很少只有一个正确答案，因此很难用程序评测。大语言模型天然适合为输出评分。我们使用一个大语言模型裁判，按照评分标准评估每份输出：事实准确性（主张是否与来源一致？）、引用准确性（引用的来源是否对应所述主张？）、完整性（是否涵盖了用户要求的所有方面？）、来源质量（是否优先使用一手来源，而非质量较低的二手来源？），以及工具效率（是否以合理的次数使用了正确工具？）。我们曾尝试用多个裁判分别评估各个部分，但发现，用一个提示词进行一次大语言模型调用，输出 0.0—1.0 的分数和通过或不通过的判定，结果最一致，也最符合人类判断。当评测用例确实有明确答案时，这种方法尤其有效，因为只需让大语言模型裁判检查答案是否正确即可，例如是否准确列出了研发预算最高的三家制药公司。使用大语言模型作为裁判，使我们能够规模化评估数百份输出。

人工评测能发现自动化遗漏的问题。测试 Agent 的人会发现评测没有覆盖的边界情况，包括异常查询上的幻觉回答、系统故障，或信息来源选择中细微的偏差。在我们的案例中，人工测试人员发现，早期 Agent 总是选择经过搜索引擎优化的内容农场，而不选择学术 PDF 或个人博客等权威但搜索排名较低的来源。在提示词中加入评估来源质量的启发式方法，有助于解决这个问题。即便自动化评测已经普及，手动测试依然不可或缺。

多 Agent 系统会产生涌现行为，也就是未经专门编程却自行出现的行为。例如，对主 Agent 的微小改动，就可能以不可预测的方式改变子 Agent 的行为。要取得成功，需要理解交互模式，而不只是单个 Agent 的行为。因此，对这些 Agent 最有效的提示词，不只是严格的指令，更是规定了分工、解决问题的方法和投入预算的协作框架。做好这件事，需要精心设计提示词和工具、可靠的启发式方法、可观测性，以及紧密的反馈循环。我们系统中的提示词示例，参见 Cookbook 中的开源提示词。

生产可靠性与工程挑战

在传统软件中，一个缺陷可能破坏某项功能、降低性能，或导致服务中断。在 Agent 系统中，微小改动会层层传导，最终引起巨大的行为变化，因此，为必须在长期运行的进程中维持状态的复杂 Agent 编写代码，会变得格外困难。

Agent 有状态，而且错误会累积。 Agent 可以运行很长时间，并在大量工具调用之间持续维护状态。这意味着我们需要让代码能够持久可靠地执行，并处理沿途发生的错误。如果没有有效的缓解措施，轻微的系统故障对 Agent 也可能是灾难性的。出错后，我们不能简单地从头重启：重新开始既昂贵，也会令用户沮丧。因此，我们构建了能够从 Agent 出错时所在位置继续运行的系统。我们还利用模型的智能来妥善处理问题：例如，告知 Agent 某个工具正在发生故障，并让它自行调整，效果出乎意料地好。我们把基于 Claude 的 AI Agent 所具有的适应能力，与重试逻辑、定期检查点等确定性的保障机制结合起来。

新方法有助于调试。 Agent 会动态决策，即使提示词完全相同，各次运行也不具有确定性。这让调试更加困难。例如，用户会反馈 Agent“找不到显而易见的信息”，但我们看不到原因。是搜索查询写得不好？选择的来源质量差？还是工具发生了故障？加入完整的生产执行轨迹记录后，我们才能诊断 Agent 为什么失败，并系统地修复问题。除了标准的可观测性，我们还监测 Agent 的决策模式和交互结构，同时不监测单次对话的内容，以维护用户隐私。这种较高层次的可观测性，帮助我们诊断根因、发现意外行为，并修复常见故障。

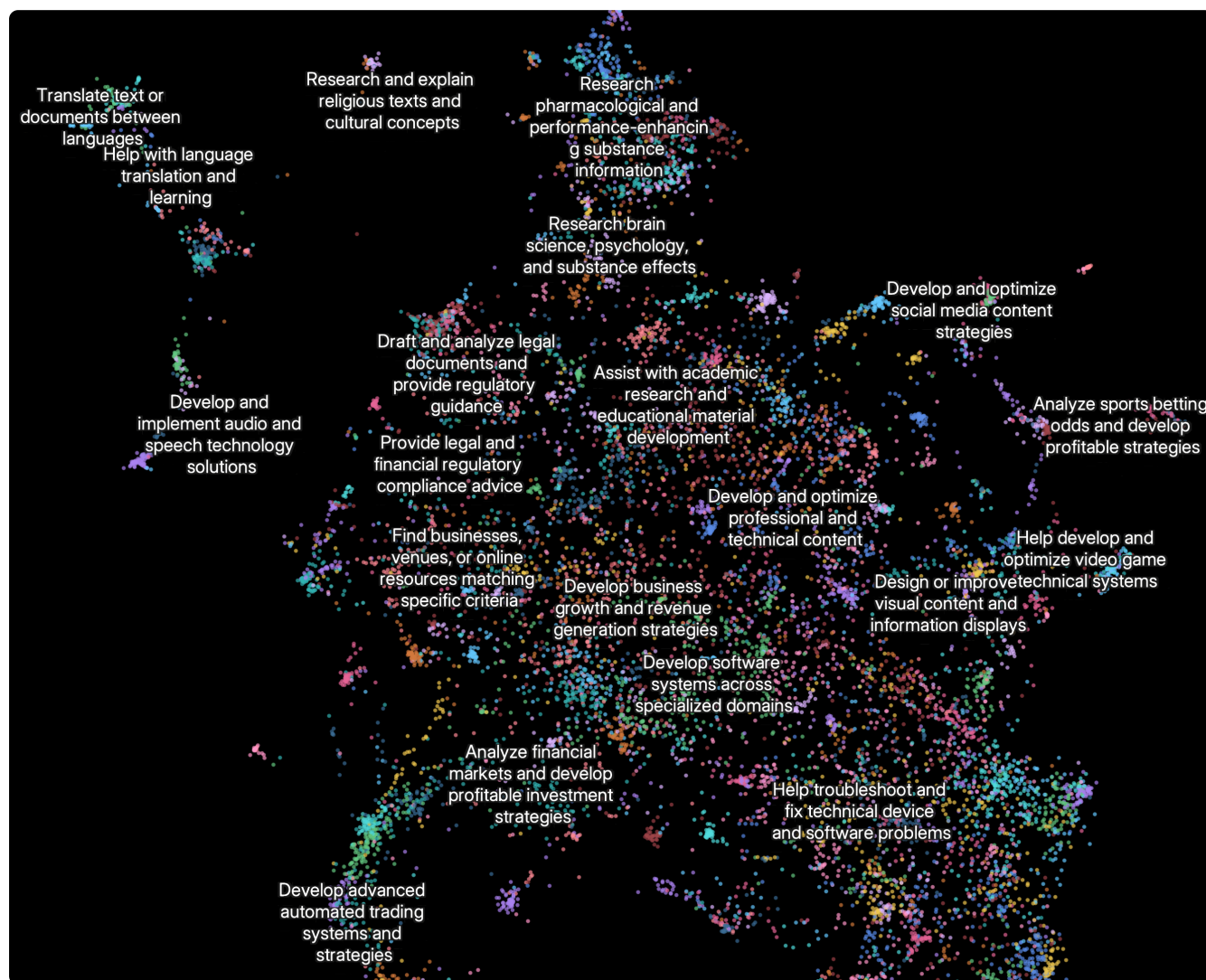
部署需要谨慎协调。 Agent 系统由提示词、工具和执行逻辑组成，是高度依赖状态、几乎持续运行的网络。这意味着，每次部署更新时，Agent 都可能处于执行过程中的任意位置。因此，我们需要避免出于好意的代码改动破坏正在运行的 Agent。我们无法同时把每个 Agent 都更新到新版本，而是采用彩虹部署：让新旧版本同时运行，并逐步将流量从旧版本切换到新版本，以避免干扰正在运行的 Agent。

同步执行会造成瓶颈。目前，主 Agent 以同步方式执行子 Agent，每次都要等一组子 Agent 全部完成后才继续。这简化了协调，但也在 Agent 之间的信息流动上造成了瓶颈。例如，主 Agent 无法引导子 Agent，子 Agent 之间无法协调，整个系统还可能因为等待某一个子 Agent 完成搜索而阻塞。异步执行可以进一步提高并行度：让 Agent 并发工作，并在需要时创建新的子 Agent。但这种异步性也会增加子 Agent 之间结果协调、状态一致性和错误传播方面的挑战。随着模型能够处理时间更长、更复杂的研究任务，我们预计性能收益将足以抵偿这些复杂性。

结语

构建 AI Agent 时，最后一段路往往占据了整个旅程的大部分。在开发者机器上能运行的代码库，需要大量工程工作才能变成可靠的生产系统。Agent 系统中的错误具有累积效应，因此在传统软件中只是小问题的状况，也可能让 Agent 彻底偏离轨道。一个步骤失败，就可能使 Agent 探索完全不同的路径，带来不可预测的结果。出于本文所述的种种原因，原型与生产系统之间的差距，往往比预想的更大。

尽管存在这些挑战，多 Agent 系统已证明自己在开放式研究任务中的价值。用户表示，Claude 帮助他们发现了此前未曾考虑的商业机会，理清了复杂的医疗选择，解决了棘手的技术缺陷，还通过发现他们独自研究时找不到的联系，最多节省了数天的工作。借助精心的工程设计、全面测试、注重细节的提示词与工具设计、稳健的运维实践，以及研究、产品和工程团队之间的紧密协作，多 Agent 研究系统可以在大规模运行中保持可靠；而这些团队需要深入理解当前 Agent 的能力。我们已经看到，这类系统正在改变人们解决复杂问题的方式。



A Clio embedding plot showing the most common ways people are using the Research feature today. The top use case categories are developing software systems across specialized domains (10%), develop and optimize professional and technical content (8%), develop business growth and revenue generation strategies (8%), assist with academic research and educational material development (7%), and research and verify information about people, places, or organizations (5%).

致谢

本文由 Jeremy Hadfield、Barry Zhang、Kenneth Lien、Florian Scholz、Jeremy Fox 和 Daniel Ford 撰写。这项工作凝聚了 Anthropic 多个团队的共同努力，是他们让 Research 功能成为现实。特别感谢 Anthropic 应用工程团队，他们的投入让这个复杂的多 Agent 系统得以投入生产。我们也感谢早期用户提供的宝贵反馈。

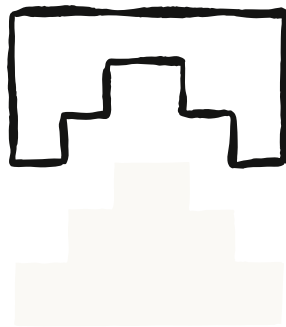
附录

下面补充一些关于多 Agent 系统的实用建议。

对经过多轮交互修改状态的 Agent 进行最终状态评测。评测在多轮对话中修改持久状态的 Agent，面临独特的挑战。与只读研究任务不同，每个动作都可能改变后续步骤所处的环境，产生传统评测方法难以处理的依赖关系。我们发现，把重点放在最终状态评测而非逐轮分析上，效果很好。与其判断 Agent 是否遵循了某个特定过程，不如评估它是否达到了正确的最终状态。这种方法承认 Agent 可能通过不同路径实现同一目标，同时仍然确保它交付预期结果。对于复杂工作流，应把评测拆成若干独立检查点，检查到这些位置时是否已发生特定状态变化，而不是试图验证每一个中间步骤。

长程对话管理。生产环境中的 Agent 经常参与长达数百轮的对话，需要周密的上下文管理策略。随着对话延长，标准上下文窗口会不够用，因而需要智能压缩和记忆机制。我们实现了这样的模式：Agent 先总结已完成的工作阶段，将关键信息存入外部记忆，然后再推进新任务。接近上下文上限时，Agent 可以创建上下文干净的新子 Agent，并通过精心交接保持工作连续性。此外，它们还可以从记忆中检索研究计划等已存储的上下文，而不会在到达上下文上限时丢失之前的工作。这种分布式方法既能防止上下文溢出，也能在长时间交互中维持对话的连贯性。

让子 Agent 把输出写入文件系统，尽量减少“传话游戏”造成的信息失真。对于某些类型的结果，子 Agent 可以直接输出，绕过主协调者，同时提高保真度和性能。不要要求子 Agent 通过主 Agent 传递所有内容，而应实现产物系统，让专门的 Agent 创建能够独立持久保存的输出。子 Agent 调用工具，把工作成果存储到外部系统，再将轻量的引用传回协调者。这可以避免多阶段处理中的信息损失，并减少在对话历史中复制大段输出所带来的 token 开销。这种模式尤其适用于代码、报告或数据可视化等结构化输出：子 Agent 的专用提示词所产生的结果，比经过通用协调者再处理后的结果更好。



想了解更多？

[探索课程](#)